ANSIBLE

AUTOMATION MADE SIMPLE

CONFIGURE • DEPLOY • AUTOMATE
YOUR INFRASTRUCTURE

"Automate Today for a Better Tomorrow"

NEERAJ SINGH
DEVOPS

Simple.
Agentless.
Powerful.
Open Source.

ANSIBLE

Infrastructure Automation for Everyone



CONFIGURATION MANAGEMENT



APPLICATION DEPLOYMENT



TASK AUTOMATION



CLOUD PROVISIONING

"Less Manual Work → More Innovation"

FLEXIBLE | SCALABLE | COMMUNITY DRIVEN

ANSIBLE – INTRODUCTION

1 What is Ansible ?

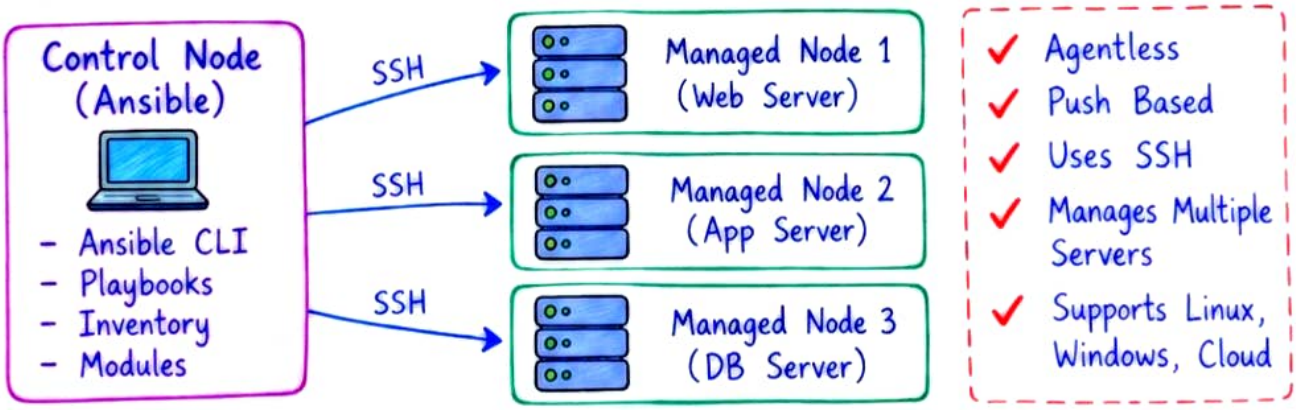
Ansible is an **open source** automation tool used for configuration management, application deployment, task automation and orchestration. It helps you **manage multiple servers** from a single control node without installing any agent on target servers **(Agentless)**

2 Why Use Ansible ?

- ✔ Simple and easy to learn.
- ✔ Agentless (uses SSH).
- ✔ Automates repetitive manual tasks.
- ✔ Consistent and reliable deployments.
- ✔ Supports configuration management and orchestration.
- ✔ Idempotent (makes only required changes).
- ✔ Large community support (open source).

> "Automate Infrastructure for a Better Tomorrow"

3 Ansible Architecture



4 Important Terminology

- **Control Node** : System where Ansible is installed and from where you run commands.
- **Managed Node** : Target servers that Ansible manages.
- **Inventory** : List of managed nodes (servers).
- **Playbook** : YAML file containing automation tasks.
- **Play** : A set of tasks to be executed on hosts.
- **Task** : A single action (uses a module).
- **Module** : Reusable unit of code to perform a specific task.
- **Role** : A way to organize playbooks, files, templates and variables.

5 Quick Tip

Ansible is **AGENTLESS** and uses **SSH** for communication.

ANSIBLE

INSTALLATION & FIRST SETUP

1 Install Ansible on Linux (Control Node)

Ansible can be installed using package manager (dnf/yum/apt) or using pip.

For RHEL / CentOS / Amazon Linux

```
# Install EPEL repository
sudo yum install -y epel-release

# Install Ansible
sudo yum install -y ansible

# Verify installation
ansible --version
```

For Ubuntu / Debian

```
# Update package list
sudo apt update

# Install Ansible
sudo apt install -y ansible

# Verify installation
ansible --version
```

> "Install Once Automate Everything"

2 Verify Ansible Installation

```
ansible --version
```

Expected Output (sample):

```
ansible [core 2.15.0]
  config file = None
  python version = 3.9.16
  executable location = /usr/bin/ansible
```

3 Setup SSH Key for Passwordless Authentication

```
# Generate SSH key (if not already created)
ssh-keygen -t rsa -b 4096

# Copy public key to managed node
ssh-copy-id user@<server-ip>
```

Ansible uses SSH for communication, so passwordless access is required.

4 Create Inventory File

```
# Create inventory file
sudo nano /etc/ansible/hosts

# Add managed nodes
[web]
web1  ansible_host=192.168.1.10
web2  ansible_host=192.168.1.11
[db]
db1   ansible_host=192.168.1.20
```

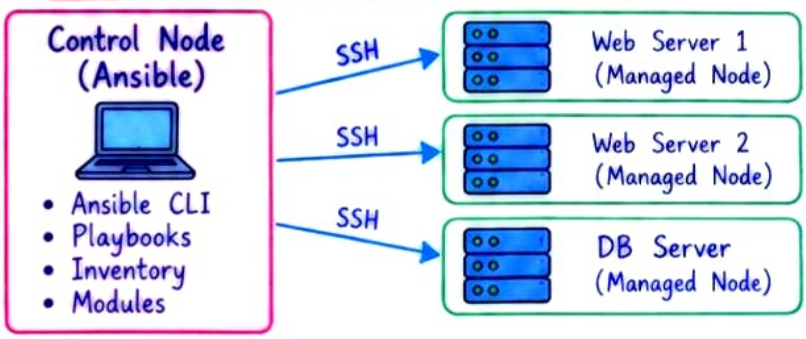
5 Test Connectivity

```
# Ping all servers in inventory
ansible all -m ping

# Ping a specific group
ansible web -m ping

# Check inventory details
ansible-inventory --list
```

6 Setup Complete – Architecture



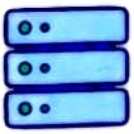
Key Points

- ✓ Ansible is installed only on Control Node.
- ✓ Uses SSH for communication.
- ✓ No agent required on managed nodes.
- ✓ Inventory contains list of servers.
- ✓ Test connectivity before running playbooks.

ANSIBLE
INVENTORY

Inventory Defines Where to Work ☺

1 What is Inventory ?



- Inventory is a file that contains a list of managed nodes (servers)
- On which Ansible will run commands, playbooks and modules.
- It can be a static file or dynamic (e.g. from AWS, Azure, etc.).

2 Types of Inventory

- **Static Inventory** : Manually managed file (INI or YAML).
- **Dynamic Inventory** : Automatically generated (e.g. AWS EC2, Azure, etc.).

3 Static Inventory (INI Format)

```
[web]
 web1  ansible_host=192.168.1.10
 web2  ansible_host=192.168.1.11
[db]
db1    ansible_host=192.168.1.20
```

Explanation :

- Groups are defined using [group_name].
- Hosts are added under each group.
- You can create child groups using [parent:children].
- You can also define variables like ansible_host, ansible_user, etc.

4 Inventory in YAML Format

```
all:
  children:
    web:
      hosts:
        web1:
          ansible_host: 192.168.1.10
        web2:
          ansible_host: 192.168.1.11
    db:
      hosts:
        db1:
          ansible_host: 192.168.1.20
```

Explanation :

- YAML format is more structured and readable.
- Useful for complex environments.
- Supports groups, child groups and variables.



More Readable & Structured

5 Group Variables and Host Variables

- group_vars/ : Variables for a group (e.g. web, db).
- host_vars/ : Variables for a specific host.
- Variables can be defined in YAML files.

Example : group_vars/web.yml

```
app_port: 8080
env: production
app_package: nginx
```

6 Dynamic Inventory (AWS EC2)

```
# Install AWS dynamic inventory plugin
pip install boto3
# Example command
ansible-inventory -i aws_ec2.yml --list
```

Explanation :

- Dynamically fetches list of EC2 instances.
- No need to manually update inventory.
- Uses AWS credentials and filters (tags, regions, etc.).

7 Useful Commands

```
ansible-inventory --list      # Show inventory in JSON format
ansible-inventory --graph     # Show inventory in tree format
ansible -m ping               # Test connection to web group
```

Key Points

- Inventory defines the target servers.
- Supports both static and dynamic sources.
- Use group_vars and host_vars for reuse.
- Dynamic inventory is widel used in cloud (AWS).

Tip : Inventory helps you manage multiple environments (Dev, QA, Prod) easily.

ANSIBLE

AD-HOC COMMANDS & MODULES

> "Ad-hoc Commands are best for quick tasks, but for complex and reusable automation use Playbooks."

1 What are Ad-Hoc Commands ?

- Ad-hoc commands are one-liner Ansible commands used to perform quick tasks on managed nodes without writing a playbook.
- They are useful for simple and immediate tasks.
- Best for testing, troubleshooting and small automation tasks.



2 Basic Ad-Hoc Command Syntax

```
ansible <host-pattern> -m <module> -a "<arguments>"
```

Example :

```
ansible all -m ping
```

3 Commonly Used Modules with Examples

Module	Purpose	Example Command
ping	Check connectivity to hosts	ansible all -m ping
command	Run a command on remote host	ansible all -m command -a "uptime"
shell	Run a shell command (supports pipes)	ansible all -m shell -a "df -h | grep /dev/sda"
copy	Copy files to remote host	ansible web -m copy -a "src=/tmp/app.sh dest=/home/ubuntu/app.sh"
file	Manage files and directories	ansible all -m file -a "path=/tmp/test state=directory"
package	Install/Remove packages	ansible all -m package -a "name=nginx state=present"
service	Manage services	ansible all -m service -a "name=nginx state=started"
user	Manage users	ansible all -m user -a "name=devops state=present"
template	Deploy templates (Jinja2)	ansible all -m template -a "src=app.j2 dest=/etc/app.conf"
debug	Print messages / variables	ansible all -m debug -a "msg='Hello Ansible'"

4 Command vs Shell Module

Command Module

- Runs simple commands.
- Does not process shell features like pipes, redirection, env vars.
- More secure.
- Recommended when shell features are not needed.



```
ansible all -m command -a "ls -l"
```

shell Module

- Runs commands through shell.
- Supports pipes, redirection, environment variables, etc.
- More powerful.
- Use only when shell features are required.



```
ansible all -m shell -a "ps -ef | grep nginx"
```

5 Run Ad-Hoc on Specific Hosts/Groups

- Run on all hosts
  `ansible all -m ping`
- Run on a specific group
  `ansible web -m ping`
- Run on a specific host
  `ansible web1 -m ping`
- Run with sudo (become)
  `ansible all -m command -a "whoami" -b`



6 Check Available Modules

- List all modules
  `ansible-doc -l`
- Get help for a module
  `ansible-doc copy`
- Show module examples
  `ansible-doc service -s`



Key Points

- Use ad-hoc for quick and small tasks.
- Modules are idempotent (safe to run multiple times).
- Prefer playbooks for complex workflows.
- Use --check for dry run (where supported).
- Explore more modules using ansible-doc.

> "Right Module for the Right Task Makes Automation Easy"



Ad-hoc commands are useful, but real automation comes from Playbooks and Roles.

ANSIBLE
PLAYBOOKS & YAML

> "Write Once Automate Everywhere"

1 What is a Playbook ?

- A playbook is a **YAML file** that contains a set of plays.
- Each play defines a group of hosts and a list of tasks to be executed on those hosts using Ansible modules.
- Playbooks are used for configuration management, application deployment and automation.



2 Basic Playbook Structure

```yaml
---
- name: Example Playbook
  hosts: web
  become: yes

  tasks:
    - name: Install Nginx
      package:
        name: nginx
        state: present

    - name: Start Nginx Service
      service:
        name: nginx
        state: started
        enabled: yes
```

Explanation :

- `---` : YAML document start
- `name` : Name of the play
- `hosts` : Target hosts or group
- `become` : Run tasks with sudo privileges
- `tasks` : List of tasks to execute
- `name` : Name of the task
- `module` : Ansible module (e.g. package, service)
- `state` : Desired state (present, started, etc.)

Run Playbook :



```
# Run the playbook
ansible-playbook site.yml
# Run with verbose output
ansible-playbook site.yml -v
```

3 Key Playbook Elements

Element	Description
Play	Defines a set of tasks for a group of hosts.
Task	A single action using a module.
Module	Performs a specific operation (e.g. package, service).
Inventory	Defines the target hosts.
Variables	Store values for reuse.
Handlers	Tasks that run only when notified.
Roles	Organize playbooks into reusable components.

4 Multiple Plays Example

```yaml
---
- name: Install and Configure Nginx
  hosts: web
  become: yes
  tasks:
    - name: Install nginx
      package: { name: nginx, state: present }
    - name: Start nginx
      service: { name: nginx, state: started }

- name: Deploy Application
  hosts: app
  become: yes
  tasks:
    - name: Copy application files
      copy: { src: /tmp/app/, dest: /opt/app/ }
```

5 Run for Specific Components

```
# Run specific tags (see next page)
ansible-playbook site.yml --tags "nginx"
# Limit to specific hosts
ansible-playbook site.yml --limit web1
# Check syntax before running
ansible-playbook site.yml --syntax-check
# Dry run (check mode)
ansible-playbook site.yml --check
```

6 Best Practices



- ✔ Use meaningful names for plays and tasks.
- ✔ Keep playbooks modular (use roles).
- ✔ Use variables instead of hardcoding values.
- ✔ Use idempotent modules (safe to run multiple times).
- ✔ Keep playbooks in version control (Git).
- ✔ Use tags for selective execution.
- ✔ Test playbooks in lower environments (Dev/QA) first.
- ✔ Follow a clear directory structure.



Playbooks make automation **repeatable**, **consistent** and **easy to maintain**.

ANSIBLE

VARIABLES, FACTS & CONDITIONALS

> "Use Variables for Flexibility, Not Hardcoding"

1 What are Variables ?

- Variables are used to store values that can be reused in playbooks, roles or inventory.
- They help in making playbooks dynamic, flexible and easy to maintain.
- You can define variables at multiple places (playbook, vars file, inventory, etc.).

Same playbook can be used for different environments just by changing variable values.

2 Ways to Define Variables

2.1 Define in Playbook

yaml

```yaml
---
- hosts: web
  vars:
    app_name: nginx
    app_port: 8080
  tasks:
    - name: Print variable
      debug:
        msg: "App is {{ app_name }}"
```

2.2 External Variables File

vars.yml

```yaml
app_name: nginx
app_port: 8080

# Include in playbook
- hosts: web
  vars_files:
    - vars.yml
```

2.3 Inventory Variables

inventory

```
# in inventory file
[web]
web1 ansible_host=192.168.1.10
app_port=8080

# or in group_vars/web.yml
app_name: nginx
app_port: 8080
```

3 Variable Precedence (High to Low)

1. Extra variables (-e) (Highest)
2. Task variables
3. Block variables
4. Role variables
5. Play variables
6. Variables files (vars_files)
7. Inventory variables (host_vars, group_vars)
8. Facts (ansible_facts)
9. Default variables (role defaults) (Lowest)

High Priority ↓ Low Priority

4 Ansible Facts

- Ansible Facts are system information automatically collected from managed nodes.
- They are stored in ansible_facts variable.
- Useful to get OS, IP, kernel, memory, etc.

Get all facts:

```
ansible web -m setup
```

- Get specific fact:

```
ansible web -m setup -a "filter=ansible_os_family"
```

Example: Using Facts in Playbook

```yaml
- name: Print OS information
  hosts: web
  tasks:
    - debug:
        msg: "OS is {{ ansible_os_family }}"
```

5 Register Variable

- Used to store the output of a task in a variable.
- You can use this variable in later tasks.

yaml

```yaml
- name: Check disk space
  shell: df -h
  register: disk_output

- name: Show output
  debug:
    var: disk_output.stdout_lines
```

6 set_fact Module

- Used to define variables during runtime.
- Useful when you want to set or modify variables dynamically.

yaml

```yaml
- name: Set a fact
  set_fact:
    my_message: "Hello Ansible"
- name: Use the fact
  debug:
    msg: "{{ my_message }}"
```

Variables can also be created using set_fact

7 Conditions (when)

- Used to execute a task only when a condition is true.

yaml

```yaml
- name: Install nginx only on RedHat
  package:
    name: nginx
    state: present
  when: ansible_os_family == "RedHat"
```

8 Other Useful Conditionals

Condition	Description
changed_when	Define when a task is considered changed.
failed_when	Define when a task should fail.
when	Run task based on condition.
ignore_errors	Ignore failures (not recommended).
any_errors_fatal	Stop execution if any host fails.

Use variables, facts and conditions to make your playbooks dynamic, reusable and easy to maintain.

ANSIBLE

LOOPS, TEMPLATES, HANDLERS & TAGS

"Automate Repetitive Tasks the Smart Way"

1 Loops in Ansible

- Loops are used to run a task multiple times with different values.
- They help in automating repetitive tasks.

1.1 LOOP (preferred)

loop

```
- name: Install multiple packages
  hosts: web
  tasks:
    - name: Install packages
      package:
        name: "{{ item }}"
        state: present
      loop:
        - nginx
        - git
        - curl
```



1.2 with_items (legacy)

with_items

```
- name: Create multiple users
  hosts: web
  tasks:
    - name: Create users
      user:
        name: "{{ item }}"
        state: present
      with_items:
        - user1
        - user2
        - user3
```



Key Points

- loop is the latest and recommended syntax.
- with_items is legacy (still works).
- item is used to access each value.
- Can loop over lists, files, dictionaries, etc.

2 Jinja2 Templates

- Templates are used to create dynamic configuration files using variables.
- Ansible uses Jinja2 templating engine.

2.1 Template File (nginx.conf.j2)

nginx.conf.j2

```
server {
  listen {{ app_port }};
  server_name {{ server_name }};

  location / {
    proxy_pass http://{{ backend }};
  }
}
```



2.2 Playbook using Template

playbook.yml

```
- name: Deploy Nginx config
  hosts: web
  tasks:
    - name: Copy template
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        notify: Restart nginx
```

Benefits

- Dynamic and reusable files.
- Use variables for flexibility.
- Same template can be used across environments (Dev/QA/Prod).
- Avoids hardcoding values.
- Commonly used for config files like nginx, httpd, app configs, etc.

3 Handlers in Ansible

- Handlers are special tasks that run only when notified by another task (when there is a change).

3.1 Example with Handler

```
- name: Install and configure Nginx
  hosts: web
  tasks:
    - name: Copy config
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        notify: Restart nginx
  handlers:
    - name: Restart nginx
      service:
        name: nginx
        state: started
```

How Handlers Work ?

Task makes a change (e.g. config file updated)

↓

Notify Handler (Restart nginx)

↓

Handler runs at end (only if change occurred)

Key Points

- Handlers run only when notified.
- Useful for services restart.
- Multiple tasks can notify the same handler.
- Handlers run at the end of the play (not immediately).
- Commonly used for restart/reload services like nginx, httpd, tomcat, etc.

4 Tags in Ansible

Tags are used to run specific tasks from a playbook.

Example: Playbook with Tags

```
- name: Install Nginx
  hosts: web
  tasks:
    - name: Install nginx
      package:
        name: nginx
        state: present
        tags: ["install"]
    - name: Start nginx
      service:
        name: nginx
        state: started
        tags: ["start"]
```



Run with Tags

```
# Run only install tasks
ansible-playbook site.yml --tags install

# Run only start tasks
ansible-playbook site.yml --tags start

# Skip specific tags
ansible-playbook site.yml --skip-tags install
```

Key Points

- Tags help in selective execution.
- Useful for large playbooks.
- Can run or skip tags.
- Multiple tags can be used for a task.
- Common tags: install, configure, start, stop, deploy, etc.

Use loops, templates, handlers and tags to make your playbooks modular, reusable and efficient.

ANSIBLE
ROLES

"Roles Make Automation Scalable"



1 What are Ansible Roles ?

- Roles are a way to organize playbooks and related files (tasks, handlers, variables, templates, etc.) into a structured directory.
- They make playbooks modular, reusable and easy to maintain.
- Useful for managing complex automation projects.

2 Role Directory Structure

```
myrole/
├── tasks/       main.yml
├── handlers/    main.yml
├── templates/   file.j2
├── files/       app.conf
├── vars/        main.yml
├── defaults/    main.yml
└── meta/        main.yml
```

Explanation :

tasks	→	Contains main tasks for the role.
handlers	→	Contains handlers (e.g. restart service).
templates	→	Jinja2 templates.
files	→	Static files to be copied.
vars	→	Role specific variables (high precedence).
defaults	→	Default variables (low precedence).
meta	→	Metadata (dependencies, role info).

3 Create a Role

```
# Create a role named nginx
ansible-galaxy init nginx
```

What it does?
This will create the directory structure with all necessary files and folders.

4 Using a Role in Playbook

```
---
- name: Install and configure Nginx using role
  hosts: web
  become: yes
  roles:
    - nginx
```

Passing Variables to a Role :

```
---
- hosts: web
  roles:
    - role: nginx
      vars:
        nginx_port: 8080
```

5 Role Variables (defaults vs vars)

defaults/main.yml (low precedence)

```
nginx_port: 80
nginx_user: www-data
```

vars/main.yml (high precedence)

```
nginx_version: 1.24
nginx_conf_path: /etc/nginx/nginx.conf
```

Key Points :

- Use defaults for values that can be overridden.
- Use vars for fixed values.
- Playbook variables override both.

6 Role Dependencies (meta/main.yml)

```
dependencies:
  - role: common
  - role: mysql
    vars:
      mysql_version: 8.0
```

Explanation :

- Roles can depend on other roles.
- Dependencies are defined in meta/main.yml.
- Useful for reusability (e.g. common role for all services).
- Dependencies are executed before the current role.

7 Useful Commands

```
ansible-galaxy init <role_name>      # Create a new role
ansible-galaxy install <role_name>   # Install a role from Galaxy
ansible-galaxy list                  # List installed roles
```

Best Practices :

- Keep roles small and focused.
- Use meaningful names.
- Document variables in README.
- Use role dependencies for common components.
- Store roles in version control (Git).

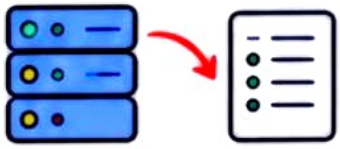
Roles make Ansible projects modular, reusable and easy to maintain at scale.

ANSIBLE
INVENTORY

> "Right Inventory Means the Right Automation at the Right Place"

1 What is Inventory ?

- Inventory is a file that contains the list of target hosts or groups on which Ansible will run tasks.
- It can be a static file or dynamic (from cloud).
- Inventory helps in managing multiple servers easily.



2 Types of Inventory

Type	Description	Example
Static Inventory	Manually create a file with host details.	inventory.ini
Dynamic Inventory	Generated from cloud providers (AWS, GCP, Azure, etc.).	aws_ec2.py
INI Format	Simple and commonly used.	hosts.ini
YAML Format	More structured and powerful.	inventory.yml

Benefits :

- ✔ Manage multiple hosts easily.
- ✔ Group servers logically.
- ✔ Support for dynamic environments (cloud).
- ✔ Flexible format (INI, YAML, dynamic).
- ✔ Makes playbooks reusable.

3 Static Inventory (INI Format)

inventory.ini

```
[web]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[db]
db1  ansible_host=192.168.1.20

[all:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_rsa
```



4 Static Inventory (YAML Format)

inventory.yml

```
all:
  hosts:
    web1:
      ansible_host: 192.168.1.10
    web2:
      ansible_host: 192.168.1.11
  children:
    db:
      hosts:
        db1:
          ansible_host: 192.168.1.20
  vars:
    ansible_user: ec2-user
    ansible_ssh_private_key_file: ~/.ssh/id_rsa
```



5 Dynamic Inventory (AWS Example)

aws_ec2.py

```
# Use AWS dynamic inventory script
ansible-inventory -i aws_ec2.py --list

# or in ansible.cfg
[inventory]
  enable_plugins = aws_ec2
  plugin = aws_ec2
  regions = ap-south-1
  keyed_groups =
    - key: tags.Role
      prefix: tag
```



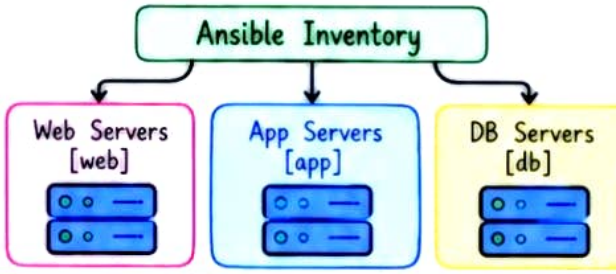
6 Useful Commands

```
# Test inventory file
ansible-inventory -i inventory.ini --list
# Ping all hosts
ansible all -i inventory.ini -m ping
# Ping specific group
ansible web -i inventory.ini -m ping
# Check host variables
ansible web1 -i inventory.ini -m setup
# Use YAML inventory
ansible all -i inventory.yml -m ping
```



7 Best Practices

- ✔ Use groups to organize hosts.
- ✔ Use meaningful names for hosts and groups.
- ✔ Keep sensitive data (keys, passwords) secure.
- ✔ Use dynamic inventory for cloud environments.
- ✔ Test the inventory before running playbooks.



A well-managed inventory makes your automation scalable and environment independent.

ANSIBLE VAULT

> Encrypt Today for a Safer Tomorrow ☺

1 What is Ansible Vault ?

- Ansible Vault is used to encrypt sensitive data such as passwords, API keys, certificates, etc. in playbooks, vars files or any Ansible file.
- It helps in keeping sensitive information secure in a Git repository.

2 Why Use Ansible Vault ?

- ✅ Keeps sensitive data secure.
- ✅ Prevent exposing credentials in Git.
- ✅ Encrypt entire files or specific variables.
- ✅ Easy to use and integrate with playbooks.
- ✅ Supports strong AES encryption.

3 Vault Commands

```
# Encrypt a new file
ansible-vault create secrets.yml

# Edit an existing encrypted file
ansible-vault edit secrets.yml

# View an encrypted file
ansible-vault view secrets.yml

# Encrypt an existing file
ansible-vault encrypt vars.yml

# Decrypt a file
ansible-vault decrypt vars.yml

# Re-encrypt with a new password
ansible-vault rekey vars.yml
```

4 Example - Encrypted Variables File

```
# create encrypted file
ansible-vault create db_credentials.yml

db_user: admin
db_password: mysecurepassword
db_host: db.example.com
```

YAML

5 Using Vault in Playbook

```
---
- name: Use encrypted variables
  hosts: web
  vars_files:
    - db_credentials.yml
  tasks:
    - name: Print DB user
      debug:
        msg: "DB User is {{ db_user }}"
```

6 Providing Vault Password

1. Prompt for password (interactive)
   ```
   ansible-playbook site.yml --ask-vault-pass
   ```
2. Use a password file
   ```
   ansible-playbook site.yml --vault-password-file vault.pass
   ```
3. Set in ansible.cfg

ansible.cfg

```
[defaults]
vault_password_file = ~/.ansible/vault.pass
```

7 Encrypt Specific Variables (Inline)

```
# Encrypt a single variable
ansible-vault encrypt_string 'mysecret' --name 'api_key'

# Output:
api_key: !vault |
      $ANSIBLE_VAULT;1.1;AES256
      663862366639333562346461613430...
```

8 Best Practices

- ✅ Use strong and unique passwords.
- ✅ Store vault password file securely (not in Git).
- ✅ Use vault for all sensitive data.
- ✅ Restrict access to vault password.
- ✅ Rotate vault passwords periodically.
- ✅ Use --vault-id for multiple vaults (advanced).

9 Common Use Cases

- ✅ Database credentials
- ✅ API keys and tokens
- ✅ SSH private keys
- ✅ Cloud provider credentials (AWS, Azure, GCP)
- ✅ SSL certificates
- ✅ Any sensitive configuration data

Use Ansible Vault to keep your infrastructure code secure and production ready.